

Gestione dei Log con lo Stack ELK

Introduzione alla Gestione dei log con ELK

Il problema. I file di log sono spesso fonte di problemi, e in un'architettura a microservizi il problema esplode: hai tantissimi servizi, ognuno su una macchina diversa, ognuno con i suoi file di log. Quando devi capire *dove* si è verificato un errore, ti ritrovi a saltare da un file all'altro su macchine diverse. Serve quindi **un posto centralizzato** dove finiscono tutti i log e **uno strumento** che ti permetta di visualizzarli, cercarli e interrogarli.

La soluzione: lo stack ELK. ELK è l'acronimo che identifica l'unione di **tre strumenti**:

E — Elasticsearch. È un **server di ricerca e analisi**, un sistema altamente scalabile che permette di gestire una grande quantità di dati con interrogazioni **quasi in tempo reale**. Proprio per questo è diventato molto popolare nel mondo dei big data: il docente cita Netflix, Facebook e tantissime altre aziende con una mole di dati consistente. I dati al suo interno vengono memorizzati in **uno o più indici**: un indice è una raccolta di documenti che presentano caratteristiche simili tra loro. In ELK, Elasticsearch è **lo storage**.

L — Logstash. Ha il compito di **aggregare i dati provenienti da varie sorgenti**. Gli arriva una marea di log, lui li raccoglie, eventualmente **li filtra** (nel nostro esempio non serve), li indicizza e **li invia direttamente a Elasticsearch**. È il "collettore".

K — Kibana. È la parte di **visualizzazione**: fornisce una **GUI** per analizzare i dati e fare query in modo molto più semplificato, invece che a colpi di chiamate REST.

Il flusso, in una riga: applicazione → Logstash (raccoglie/filtra) → Elasticsearch (memorizza/indicizza) → Kibana (visualizza).

Come si ottengono. Sono **gratuiti**: basta andare sul sito ufficiale di Elastic e scaricarli uno a uno. Il docente fa scorrere la pagina dei download e sottolinea che c'è "una marea di roba", plugin su plugin — Elasticsearch in particolare "è un mondo a parte". **Ma nel nostro caso non scaricheremo nessuno di questi software**: li tireremo su tramite immagini Docker (con Maven per la parte applicativa).

Premessa importante che fa il docente. Questo è un **video introduttivo**. Per vedere i mille casi d'uso e usare davvero questi strumenti in maniera dettagliata, rimanda sempre alla **documentazione ufficiale**. Lo scopo qui è dare una breve introduzione e metterti in condizione di iniziare a usarli.

Configurazione Spring Boot

Definizione. Questa lezione riguarda l'unica parte che tocca **il tuo microservizio**: farlo diventare una *sorgente* di log per Logstash. E la cosa interessante è che serve pochissimo — **una dipendenza e un file di configurazione**, nient'altro.

Il progetto di partenza. Un semplice progetto Spring Boot con un REST controller che ha un logger e un endpoint che non fa altro che **loggere un'informazione**. Questo è il codice reale del corso:

```
@RestController
@RequestMapping("/api")
public class HelloController {

    private static final Logger log = LoggerFactory.getLogger(HelloController.class);

    // ATTENZIONE: aggiungere dependency
    // http://localhost:8080/api/prova

    @RequestMapping(value = "/prova", method = RequestMethod.GET, produces =
"application/json")
    public String prova() {
        log.info("invocato");
        return "OK";
    }
}
```

Nient'altro: chiami `/api/prova`, lui scrive nel log la parola `invocato` e ti risponde `OK`. Il logger è lo standard **SLF4J** (`org.slf4j.Logger / LoggerFactory`), quello che usi normalmente in Spring Boot — non cambia nulla nel modo in cui scrivi i log.

Passo 1 — la dipendenza nel `pom.xml`. È la dipendenza che permette di **inviare i dati a Logstash**:

```
<dependency>
  <groupId>net.logstash.logback</groupId>
  <artifactId>logstash-logback-encoder</artifactId>
  <version>6.6</version>
</dependency>
```

Il resto del pom è un normalissimo progetto Spring Boot (`spring-boot-starter-parent 3.2.3`, `Java 17`, `spring-boot-starter-web`, `devtools` e `starter-test`). L'unica aggiunta è quella sopra.

Ricordati il *perché*, che il docente ripete più volte: **Logstash è il componente che raccoglie i dati** (in questo caso i log), eventualmente applica un filtro, e poi li spedisce dentro lo storage. Quindi il nostro progetto, come primo interlocutore, non vede Elasticsearch: **vede Logstash**.

Passo 2 — il file `logback-spring.xml`. Va creato dentro `src/main/resources`. È il file dove **configuriamo i nostri log**, e — ripete il docente — qui stiamo applicando "la configurazione più basilare possibile":

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>

    <springProperty scope="context" name="ElkSpringBoot" source="spring.application.name"/>

    <appender name="LOGSTASH"
class="net.logstash.logback.appender.LogstashTcpSocketAppender">
        <destination>localhost:4560</destination>
        <encoder class="net.logstash.logback.encoder.LogstashEncoder"/>
    </appender>

    <root level="INFO">
        <appender-ref ref="LOGSTASH"/>
    </root>

</configuration>
```

Come si legge, pezzo per pezzo:

- **<springProperty>** — qui ci mettiamo il **nome della nostra applicazione**, che è `ElkSpringBoot`.
- **<appender>** — è il cuore. Un *appender* in Logback è "dove vanno a finire i log". Qui gli stiamo dicendo: **non creare un file di log, ma invia direttamente** i nostri log tramite il `LogstashTcpSocketAppender`, cioè via **socket TCP**, all'indirizzo **localhost:4560**. L'encoder (`LogstashEncoder`) serve a serializzare l'evento di log in **JSON**, che è il formato che Logstash si aspetta.
- **<root level="INFO">** — qui inseriamo il **livello del log**: mettiamo solo il livello `info`, e l'unico appender agganciato è `LOGSTASH`.

Fine. **La configurazione lato Spring Boot è tutta qui**: una dipendenza e questo file. Nota che `application.properties` nel progetto è **vuoto** — non serve altro.

Due dettagli tecnici che il docente non approfondisce ma che vale la pena notare, visto che poi ci lavorerai: (1) l'attributo `name` di `<springProperty>` è il nome con cui la property sarà *richiamabile* dentro il file (es. `${ElkSpringBoot}`), mentre `source` è la property Spring da cui la legge — qui non viene mai usata, quindi è di fatto decorativa; (2) essendo `LOGSTASH` l'unico appender del root, **in console non vedrai**

più i log dell'applicazione: se vuoi tenerli anche a video devi aggiungere anche un appender di tipo `ConsoleAppender`.

Configurazione Logstash

Definizione. Finora abbiamo detto a Spring Boot *"manda tutti i tuoi log a quell'indirizzo"*. Ora dobbiamo dire a Logstash **cosa ricevere** e **dove mandarlo dopo**.

Dove va il file. Nello **stesso livello in cui creeremo il `docker-compose.yml`** (cioè nella root del progetto) si crea una cartella chiamata **logstash**, e dentro il file **logstash.conf**. Questo perché — lo vedremo nella lezione dopo — quella cartella verrà **montata come volume** dentro il container di Logstash.

Il file di configurazione. Ha **due macro-sezioni**: `input` (i dati in ingresso) e `output` (i dati che escono da Logstash). Questo è il file reale:

```
input {
  tcp {
    mode => "server"
    port => 4560
    codec => json_lines
  }
}

output {
  stdout {
    codec => rubydebug
  }
  elasticsearch {
    index   => "elkspringboot-logstash-%{+YYYY.MM.dd}"
    hosts   => "elasticsearch:9200"
    user     => "elastic"
    password => "changeme"
    ecs_compatibility => disabled
  }
}
```

La sezione input. Gli stiamo dicendo: *"tu riceverai delle connessioni TCP direttamente sulla porta 4560"*. `mode => "server"` significa che è Logstash a mettersi in ascolto (è il server, il nostro Spring Boot è il client). `codec => json_lines` è il formato: un evento JSON per riga — che è esattamente ciò che produce il `LogstashEncoder` lato Logback. **Questa porta 4560 è la stessa che abbiamo scritto in `logback-spring.xml`:** è il punto di aggancio tra i due mondi.

La sezione output. Ricordiamoci sempre che, una volta ottenuti i dati da Spring Boot, **Logstash li invia a Elasticsearch**. Qui gli stiamo dicendo *a chi* mandarli:

- **`stdout { codec => rubydebug }`** — stampa anche a video, in forma leggibile, quello che passa. È comodissimo in fase di debug perché nei log del container di Logstash vedi gli eventi mentre arrivano.
- **`elasticsearch { ... }`** — l'output vero. Gli diciamo di **creare un indice** chiamato `elkspringboot-logstash-` seguito dal **formato data anno.mese.giorno** (`%{+YYYY.MM.dd}`). Questo è il pattern classico: **un indice nuovo ogni giorno**, così puoi ruotare/cancellare i log vecchi per data.
- **`hosts => "elasticsearch:9200"`** — è l'indirizzo su cui inviare le informazioni. Elasticsearch, come vedremo, è in ascolto sulla **porta 9200**. Nota bene: l'host non è `localhost` ma **`elasticsearch`**, cioè il **nome del servizio** nel docker-compose — è il meccanismo delle reti Docker che avete già visto nella lezione precedente (il nome del servizio fa da hostname).
- **`user / password`** — sono username e password **standard** di Elasticsearch (`elastic / changeme`, quest'ultima impostata nel compose).

Il docente lo dice esplicitamente: **questa è la configurazione basica**. Logstash ha anche una sezione `filter` (per parsare, arricchire, scartare eventi) che qui non serve.

Configurazione Docker – Compose

Definizione. Invece di scaricare e installare a mano Elasticsearch, Logstash e Kibana, li avviamo tutti e tre come container con un unico `docker-compose.yml`. Il docente qui va veloce perché **dà per scontato Docker Compose** (è l'argomento della sezione precedente del corso): servizi, image, ports, environment, volumes, networks, depends_on.

Il file, per intero:

```
version: '3.7'

services:
  elasticsearch:
    container_name: elasticsearch
    image: docker.elastic.co/elasticsearch/elasticsearch:7.15.2
    volumes:
      - type: volume
```

source: elasticsearch

target: /usr/share/elasticsearch/data

ports:

- "9200:9200"

- "9300:9300"

environment:

ES_JAVA_OPTS: "-Xmx256m -Xms256m"

ELASTIC_PASSWORD: changeme

discovery.type: single-node

networks:

- elk

logstash:

container_name: logstash

image: docker.elastic.co/logstash/logstash:7.15.2

volumes:

- ./logstash:/usr/share/logstash/pipeline:ro

ports:

- "5044:5044"

- "5000:5000/tcp"

- "5000:5000/udp"

- "9600:9600"

- "4560:4560"

environment:

- LS_JAVA_OPTS=-Xmx256m -Xms256m

networks:

- elk

depends_on:

- elasticsearch

kibana:

container_name: kibana

image: docker.elastic.co/kibana/kibana:7.15.2

ports:

```
- "5601:5601"
```

```
environment:
```

```
- ELASTICSEARCH_URL=http://elasticsearch:9200  
- ELASTICSEARCH_USERNAME=elastic  
- ELASTICSEARCH_PASSWORD=changeme
```

```
networks:
```

```
- elk
```

```
depends_on:
```

```
- elasticsearch
```

```
networks:
```

```
elk:
```

```
driver: bridge
```

```
volumes:
```

```
elasticsearch:
```

Servizio elasticsearch. Gli diciamo: crea un container che si chiama `elasticsearch`, usa **questa versione** dell'immagine, crea il **volume** (con la configurazione di dove memorizzare le informazioni: il volume `elasticsearch` montato su `/usr/share/elasticsearch/data`, così i dati sopravvivono al riavvio del container). Poi il **mapping delle porte 9200 e 9300** (9200 è l'API REST, 9300 la comunicazione interna fra nodi del cluster). Poi una serie di **variabili d'ambiente**: `ES_JAVA_OPTS` limita la memoria della JVM a 256 MB, `ELASTIC_PASSWORD` imposta la password dell'utente `elastic`, `discovery.type: single-node` dice a Elasticsearch di partire come nodo singolo senza cercare altri nodi del cluster. Infine si collega alla **rete elk**.

Servizio logstash. Stessa versione dell'immagine — e qui il docente fa un **avvertimento importante**: *è importante che abbiano tutte la stessa versione (7.15.2), perché altrimenti non è detto che funzionino correttamente*. Poi il volume: `./logstash:/usr/share/logstash/pipeline:ro` — è la cartella che abbiamo creato nella lezione 3, montata in sola lettura (`ro`) nella cartella `pipeline` del container: è così che il nostro `logstash.conf` finisce dentro Logstash. Poi il mapping delle porte (tra cui **4560**, la nostra) e le variabili d'ambiente, dove in particolare gli diciamo **quanta memoria dedicargli** (`LS_JAVA_OPTS`). Infine: collegati alla rete `elk` e **ricordati che dipendi dall'avvio del container elasticsearch** (`depends_on`).

Servizio kibana. Nome container, stessa versione 7.15.2, mapping della porta **5601**, e nelle variabili d'ambiente **username, password e l'indirizzo di Elasticsearch** (`http://elasticsearch:9200`, di nuovo per nome di servizio). Anche lui sulla rete `elk` e anche lui **dipende da Elasticsearch**.

In fondo: si configura il **network e lk** (driver bridge) e il **volume elasticsearch**.

Da notare come si incastra il tutto: **Logstash gira in un container**, il tuo **Spring Boot gira fuori da Docker** (lo avvii da Eclipse). Per questo in `logback-spring.xml` la destinazione è `localhost:4560`: dal punto di vista dell'applicazione, la porta 4560 è quella pubblicata sull'host dal container di Logstash. Invece *dentro* la rete Docker, Logstash raggiunge Elasticsearch con il nome `elasticsearch`.

Avvio Esempio Completato

Obiettivo. Mettere insieme tutti i pezzi e vedere un log scritto da Spring Boot comparire dentro Kibana.

Passo 1 — aprire il prompt nella cartella del progetto. Il docente apre l'esploratore file sulla cartella del progetto Spring Boot (dal menù di Eclipse), e da lì — scrivendo **cmd** nella barra degli indirizzi di Windows e premendo invio — apre un prompt MS-DOS **già posizionato nella cartella del progetto**. Piccolo trucco pratico, ma è quello che serve: il `docker-compose.yml` sta lì.

Passo 2 — avviare lo stack.

```
docker-compose up
```

Stampa tutti i vari container e resta in attesa che finisca l'avvio. Nel terminale scorrono **i log di tutti i container insieme**: il docente nota che ci si potrebbe spostare fra i log dei singoli container, ma in questo esempio resta attaccato a `compose app` senza scollegarsi, così li vede tutti. **L'avvio richiede un po' di tempo**, perché sta avviando tre software che occupano parecchia memoria — in particolare Elasticsearch.

Passo 3 — avviare il microservizio e generare un log. Con lo stack su, si avvia l'applicazione Spring Boot e si chiama l'endpoint:

```
http://localhost:8080/api/prova
```

Risponde OK → l'applicazione funziona, e nel frattempo ha eseguito `log.info("invocato")`, quindi ha **spedito il log a Logstash**, che l'ha girato a Elasticsearch.

Passo 4 — verificare che l'indice sia stato creato. Ci si collega direttamente a Elasticsearch dal browser:

```
http://localhost:9200/_cat/indices?v
```

`_cat` è l'API "human readable" di Elasticsearch e `?v` aggiunge l'intestazione delle colonne. Qui compare l'elenco degli indici presenti, e fra questi **il nostro indice appena creato** (`elkspringboot-logstash-<data>`). Se l'indice c'è, la catena Spring Boot → Logstash → Elasticsearch funziona.

Passo 5 — collegarsi a Kibana. Kibana è in ascolto sulla porta **5601**:

```
http://localhost:5601
```

e si apre la dashboard.

Passo 6 — creare l'index pattern. Dal **menù a sinistra** → **Management** → **Stack Management** → **Kibana** → **Index Patterns** → **Create index pattern**. Si scrive il nome dell'indice (quello di Elasticsearch, cioè `elkspringboot-logstash-*`) e — passaggio che il docente sottolinea — gli si dice di usare anche il **timestamp** come campo tempo, *"così ci mostra anche l'orario"*. Poi si crea l'indice.

Il concetto da fissare: l'**indice** vive in Elasticsearch, l'**index pattern** è l'oggetto di Kibana che dice *"guarda quegli indici lì"*. Sono due cose diverse, e senza il secondo Kibana non sa cosa mostrarti.

Passo 7 — Discover. Si va su **Discover** e finalmente si vedono **le nostre loggate**: i log del nostro servizio REST. Cliccando su un documento si vede il dettaglio — il **timestamp**, il **logger** (`HelloController`), il **livello** (`INFO`) e il **messaggio**: `invocato`. Ed è qui che si capisce a cosa serve tutto il giro: si possono fare **query**, aggiungere **filtri**, analizzare tutto quello che interessa, "una marea di roba".
